White-Box Micro-Adaptive Query Processing

Jack Pearce
Imperial College London
jack.pearce22@imperial.ac.uk

Hubert Mohr-Daurat
Imperial College London
h.mohr-daurat19@imperial.ac.uk

Holger Pirk
Imperial College London
hlgr@ic.ac.uk

ABSTRACT

Operator performance in in-memory data management systems often suffers from micro-architectural hazards such as cache misses and branch mispredictions. While many operators have alternative implementations that are robust against such hazards, these generally perform worse when no hazards are encountered. Unfortunately, hazards are caused by order-dependent data characteristics that query optimizers cannot easily capture (sortedness, clusteredness, auto-correlation) making a priori hazard-conscious optimization difficult. In addition, statically optimized plans fail to adapt when data characteristics vary within a table.

To address both of these problems, we propose a hazard-adaptive approach to query execution. Through hardware-assisted runtime profiling of low-level metrics, operators dynamically adapt to "hazardous" data. We propose an architecture for hazard-adaptive operators and integrate our approach into a real-world database management system. We demonstrate that using hazard-adaptive operators provides a ~2-20× speedup across several TPC-H queries.

PVLDB Reference Format:
Jack Pearce, Hubert Mohr-Daurat, and Holger Pirk. White-Box Micro-Adaptive Query Processing. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at URL_TO_YOUR_ARTIFACTS.

1 INTRODUCTION

Analytical query performance is determined by two factors: queries and data characteristics. Queries determine the operations to be performed, and the optimizations that can be applied. Data characteristics determine the performance of these operations. As the number of operators is usually moderate, capturing query characteristics (such as selection/join predicates or grouping keys) of a dataflow execution engine is comparatively easy. Accurately formalizing data characteristics, however, is significantly harder.

Approaches for capturing data characteristics are typically tailored to logical query optimization, i.e., the order of relational operators within a query. The data characteristics used to achieve this do not capture the ordering of the data[1], i.e. they are order-invariant. For example, simple histograms (usually built on sampled values) suffice to estimate single-attribute cardinalities required for join or selection-order optimization. Furthermore, to capture complications like correlation, histograms are multidimensional and samples larger, though the principle remains the same.

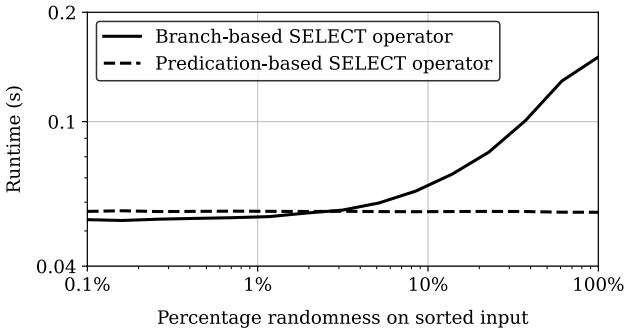


Figure 1: Runtime of SELECT index generation at a fixed 50% selectivity for 25M 64-bit integers on a conventional server

However, query processing performance is purely determined by order-invariant metrics, such as selectivity, but highly affected by patterns in the data (e.g., sortedness, clustering, auto-correlation). This includes the selection of the optimal (physical) algorithm to implement a logical operator. To illustrate this, consider Figure 1 which shows the effect of the sortedness of input data on the performance of two well-known implementations of the selection operator: one branching and one branch-free (both described by Ross [32]). The experiment shows that, on this machine[2] at equal selectivity, the sortedness of the data can have a 3× performance impact on the operator. We found that other operators (e.g., group, join, intersect) suffer from similar problems.

Capturing auto-correlation such as sortedness, clusteredness, or other patterns in the data is challenging. As histograms are order-invariant, they do not capture auto-correlation, and extending them to do so is far from trivial [26]. Other approaches to measuring the sortedness of data tend to be complicated and brittle [1, 5, 16]. Any approach based on these techniques would inherit these traits.

An alternative approach would be to not quantify auto-correlation a-priori, but implement operators that adapt their behavior at runtime on a batch-by-batch basis. This is conceptually related to the paradigm of "micro-adaptivity" which Răducanu et al. [33] introduce as "a framework that keeps many alternative function implementations in a system". This addresses two problems: first, it makes operator robust against sub-optimal optimizer decisions. Second, micro-adaptive operators change their implementation to make the best use of regions of locality within a single dataset which addresses a known shortcoming of classic optimizers.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

[1]The notable exception to this is the maintenance of "interesting orders" for opportunistic use of a merge-join or merge-aggregation optimization .

[2]We observed up to a 10× performance impact on other machines.

However, Răducanu et al. propose a black-box micro-adaptive approach to select the optimal assembly code implementation at runtime: a test and learn framework using execution time to capture utility/reward [33]. This approach relies only on a common operator interface, allowing implementation details to be abstracted. However, it requires every implementation to be continuously tested to select the best, thus limiting this approach to simple cases (like using different compilers). Further, if data locality varies within a table the optimal implementation, this must be recognized and the adaptivity-algorithm restarted, resulting in high overhead [17, 33].

To address these shortcomings, we propose micro-adaptive operators which are "white-box"[3] in a system that continuously monitors the associated micro-architectural hazards and "indicates" when a switch to an alternative implementation would be beneficial. To this end, we make the following technical contributions:

- We present an architecture for a hazard-adaptive data management system, including hazard-adaptive white box operators that maintain state across operator implementations.
- We develop a class of algorithms that are hazard-adaptive versions of classic data-processing algorithms. We exemplarily apply this to three operators: `SELECT`, `GROUP`, and `JOIN`. This includes a simple, elegant, effective extension of radix partitioning that utilizes sideways information passing between the input tables to achieve adaptivity based on the locality in both tables.
- To demonstrate practicality we integrate the hazard-adaptive operators into a real-world database management system to act as a virtually overhead-free kernel accelerator. We evaluate the performance across a range of TPC-H queries and observe ~2-20× performance improvements.

We begin by establishing background knowledge in Section 2 before presenting the hazard-adaptive architecture in Section 3 and creating concrete operators in Section 4. In Section 5 we extend the architecture to co-adaptive operators. Section 6 presents the overall system architecture. We evaluate the system's performance, discuss related work, and conclude in Sections 7, 8 and 9, respectively.

2 BACKGROUND

Before introducing the hazard-adaptive architecture let us provide essential background.

2.1 Micro-Architectural Hazards

Micro-architectural hazards are defined as "situations that prevent the next instruction in the instruction stream from executing during its designated clock cycle" [18]. To develop high-performing, micro-architecturally efficient algorithms, it is crucial to minimize these hazards. This minimizes the resulting number of stall cycles and maximizes pipeline utilization. The three classes of hazards are:

(1) Control hazards are caused by control-flow changes (branches or jumps). They lead to stall cycles and subsequent pipeline flushes due to bad speculation. Stall cycles can also be caused by a lack of eligible instructions due to control-flow dependencies.

(2) Data hazards occur when the instruction pipeline is stalled due to operands not being ready at execution. Such stalls result from cache or TLB misses when data retrieval from main memory is slower than the speed at which a CPU processes that data.

(3) Structural hazards stem from limited CPU resources resulting in multiple instructions requiring access to the same hardware resource simultaneously. For example, an ALU stall can occur when executing resource-intensive instructions.

2.2 Micro-Architecturally Efficient Algorithms

Micro-architecturally efficient algorithms maximize CPU efficiency by minimizing hazards. The principles underpinning these algorithms fall into four categories [18]:

(1) Data Locality and Cache Awareness: Maximizing cache utilization minimizes data hazards and prevents data stalls. This can be achieved by increasing data locality to prevent unnecessary cache misses, increasing cache line utilization, reducing the overall cache footprint of the program, and prefetching data where possible.

(2) Minimizing Control Flow and Branches: Control hazards through bad speculation can be removed by writing branch-free code. Limiting the use of `JUMP` instructions minimizes frontend stalls caused by a lack of eligible instructions.

(3) Parallelism and Vectorization: Instruction-level parallelism can be maximized by reordering instructions to minimize dependencies and therefore maximize the utilization of execution units. The use of vectorization to execute instructions statically parallel by performing operations on multiple data elements in parallel further increases utilization of all pipeline stages, reduces instruction overhead, and improves memory bandwidth utilization.

(4) Optimizing Memory Access Patterns: A high cache miss rate is typically due to a random memory access pattern. Switching to a sequential access pattern greatly reduces the cache miss rate.

Performance engineering techniques apply these principles to increase an algorithm's robustness to hazards and thereby develop micro-architecturally efficient algorithms.

2.3 Hardware Performance Counters

Performance counters are a set of registers located within the performance monitoring unit (PMU) of modern CPUs. These counters can be configured to count micro-architectural events during program execution, enabling the profiling of software [21, 37, 38]. This makes performance counters a crucial resource for identifying and solving performance issues [15]. The key advantage of performance counters lies in their low execution cost. Malone et al. confirm that the runtime performance overhead is virtually zero [21]. This is corroborated by Zeuch et al. [39] and the Intel documentation [11].

3 HAZARD-ADAPTIVE ARCHITECTURE

As shown in Section 1, the optimal operator implementation depends on the input data characteristics and the resulting hazards. A micro-adaptive operator should use the frequency of these hazards to select the optimal implementation. Therefore, the objective is to design *hazard-adaptive* operators that switch between alternative implementations at runtime in response to changing hazard rates.

Figure 2 illustrates the architecture we used to achieve this hazard-adaptive behavior. The hazard-adaptive operator interface

[3] we are aware of the conceivable connotation of the term "white-box" but use it due to the lack of an accepted alternative

matches a non-adaptive operator, but also access performance counters to inform their hazard-adaptivity. The foundation of the hazard-adaptive operator is a number of alternative operator implementations. We refer to these as the "foundational algorithms".

The foundational algorithms are coordinated and orchestrated by a "dispatcher" which is responsible for selecting the optimal algorithm to use, i.e., the "active operator". The dispatcher dispatches micro-batches of the input vector to the active operator, combining the foundational algorithms into a single hazard-adaptive operator.

The dispatcher is directed by the "monitor" that reads the performance counters and, based on a set of heuristics, sends a signal to the dispatcher to adjust the hazard-robustness of the hazard-adaptive operator if necessary. In the subsequent sections, we will discuss each component in Figure 2 in detail.

3.1 Hazard-Affected (HA) Algorithms

Foundational algorithms form the options of alternative implementations that the dispatcher can choose between. To create these alternative implementations we distinguish two categories of algorithms, "*hazard-affected*" algorithms and "*hazard-robust*" algorithms. Hazard-adaptive operators are formed by combining a single hazard-robust algorithm and at least one hazard-affected algorithm.

The defining objective of hazard-affected algorithms is the minimization of work (typically CPU instructions executed). This is achieved by focusing on the efficiency of the algorithmic approach. As a result, the performance of this category of algorithm is dependent on the input data characteristics for two reasons: Firstly, work is only executed if the data requires it, making the number of instructions executed dependent on the data. Secondly, this frequently introduces control-flow dependencies (to determine whether to execute work) making the algorithm susceptible to control hazards. Moreover, algorithms within this category almost universally do not focus on micro-architectural efficiency, instead focusing on work efficiency. Consequently, they are susceptible to the micro-architectural hazards described in Section 2.1, the generation of which is also determined by the data characteristics. This dependency of performance on the data characteristics makes the performance of hazard-affected algorithms hard to predict.

To illustrate this, consider a branching implementation of the `SELECT` operator: `if(/*condition*/) output[j++] = input[i++];`. It has highly variable performance for both of these reasons. Firstly, the output is only generated if the selection predicate is satisfied. Secondly, the performance at a given selectivity is determined by the sortedness of the input due to control-flow dependencies and the resulting control hazards. The variable performance is illustrated in Figure 1 and means that a hazard-robust algorithm is superior under certain input data characteristics

3.2 Hazard-Robust (HR) Algorithms

The alternative category of algorithms are hazard-robust. Unlike hazard-affected algorithms, the key principle of hazard-robust algorithms is micro-architectural robustness, i.e., limited exposure to control, data, and structural hazards.

The performance of hazard-robust algorithms is far less dependent on the input data vs. hazard-affected algorithms. Their robustness to hazards ensures consistently high CPU instruction pipeline

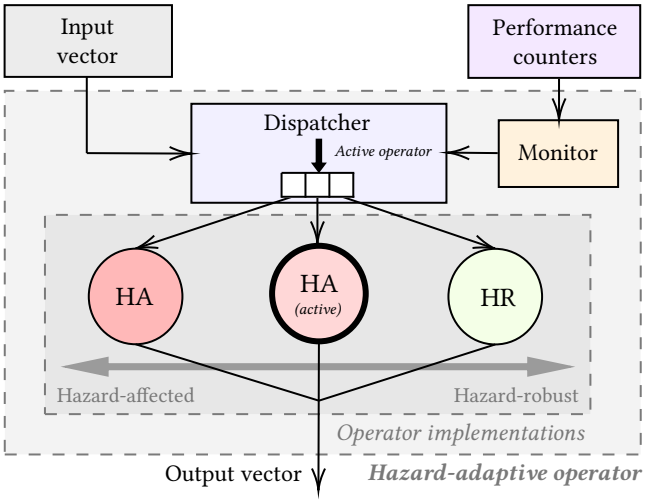


Figure 2: Hazard-adaptive operator architecture schematic

throughput, i.e., largely unaffected by data characteristics. Additionally, as we will illustrate in Section 7, hazard-robust algorithms are typically designed such that the quantity of work they perform is largely independent of data characteristics. This is illustrated by the predication-based implementation of the `SELECT` operator: `output[j] = input[i++]; j += (/*condition*/);`. Figure 1 shows that its performance is independent of the sortedness of the data.

There is substantial work on techniques that can be used to develop hazard-robust versions of hazard-affected algorithms [7, 12, 13, 27, 30, 32, 36]. However, existing data management systems typically do not include hazard-robust implementations because they sacrifice best-case performance for worst-case performance. Hazard-adaptive operators provide an architecture to make these algorithms useful by executing them only when it is beneficial.

3.3 The Monitor

To maximize best-case performance the active operator starts as the most hazard-affected implementation.

Dominant Micro-Architectural Event. To identify when to switch to a more robust algorithm the performance of the hazard-affected algorithm must be quantified. This can be achieved by the frequency of hazards since it is these hazards that degrade performance. To use a single hazard to quantify performance we define the dominant micro-architectural event as the event primarily responsible for generating the hazards that degrade the algorithm's performance.

By periodically sampling the event counter and calculating the per-tuple frequency of the dominant event, the monitor assesses the current performance of the hazard-affected algorithm.

A key advantage of this performance monitoring vs. traditional code instrumentation is that it monitors the key data characteristic that drives performance rather than a higher-level characteristic that only gives a partial view of performance. For example, in Figure 1 for `SELECT`, randomness (more precisely predictability) is the key data characteristic rather than simply selectivity. Using performance counters directly captures these patterns in the data.

```
enum Event {BranchMispredictions, LastLevelCacheMisses,
            L2TlbStoreMisses, ...};

template <class InputType, class OutputType>
class HazardAdaptiveOperator {
  HazardAdaptiveOperator(const vector<InputType> &input);
  size_t processMicroBatch(size_t n);
  void adjustRobustness(Event event, int adjustment);
  vector<OutputType> getOutput(); };
```

Listing 1: Hazard-adaptive operator interface

```
template <class InputType, class OutputType, class State>
class OperatorImplementation {
  OperatorImplementation(size_t inputSize,
                         size_t maxOutputSize);
  void updateState(State &&state);
  size_t processMicroBatch(InputType *input, size_t n);
  vector<OutputType> getOutput();
  State getState() && { return std::move(state); }
  State state; };
```

Listing 2: Foundational algorithm interface

Heuristics. To signal the dispatcher when to switch algorithms, the monitor requires a heuristic to determine the threshold event frequency where increased robustness is beneficial. This threshold is influenced by the hardware, element size (e.g. 32-bit vs. 64-bit), and the degree of parallelization (DOP). Therefore a unique heuristic is required for each combination of these factors on each machine.

These heuristics are static (at runtime) and therefore can be considered as machine constants. This is because many micro-architectural events are determined on a per-core basis, (e.g., branch mispredictions, TLB misses) isolating them from other processes or concurrent operators. Where an algorithm relies heavily on a shared resource they are naturally adaptive to contention of the resource. For example, algorithms heavily using the last-level cache (e.g., hash aggregation) will have last-level cache misses as the dominant event which will naturally account for other processes also using the last-level cache, and the monitor will increase robustness accordingly.

3.4 Dispatcher

Hazard-Adaptive Operator Interface. To ensure extensibility, we decouple the heuristics for identifying when the frequency of hazards is too high (the monitor) from the switching logic used to select the active implementation (the "dispatcher"). This allows the monitor to track multiple micro-architectural events and respond to changes in any of them allowing for the extension to more complex operators that require tracking of multiple hazards.

To communicate with the dispatcher the monitor uses the push-based hazard-adaptive operator interface in Listing 1, using the `adjustRobustness()` method to signal when increasing the algorithm robustness is beneficial. Based on `adjustment` value, the dispatcher selects the active operator and continues to process micro-batches via the push-based interface in Listing 2.

Foundational Algorithm Interface. To avoid unnecessary repeated initialization costs and ensure efficient use of intermediate data structures, foundational algorithms are encapsulated into an object that maintains these resources. For example, where memory is allocated (e.g., a hash table) it is reused for all micro-batches processed by that operator implementation. The interface for these objects in Listing 2 defines an interface for the dispatcher to interact with.

This interface allows the dispatcher to efficiently retain previous processing across algorithm switches. This is a particularly challenging aspect of micro-adaptivity when applied to more complex operators such as pipeline-breakers which require information sharing across the entire input dataset. To retain progress across switches the the results from different algorithms (that have processed different parts of the input) must be merged. As a switch was made to the hazard-robust algorithm (hence the merge), at least some of the data is not suitable for processing using a hazard-affected algorithm. Therefore, the hazard-robust algorithm is used to merge the results. If results are merged immediately when a switch is made, work is duplicated by repeatedly merging some of the same results every time a switch occurs. To prevent this, merges are deferred until after the entire input has been processed.

To efficiently merge results each foundational object has an internal state which collates results from `processMicroBatch()` calls. During a merge, this state is efficiently (copy-free) passed from the hazard-affected implementations to the hazard-robust implementation. The dispatcher uses the `getState()` method to extract the internal state and transfer it to another implementation. This method is "r-value reference qualified" (indicated by the `&&`-suffix) encoding that it can only be invoked on objects "marked for destruction" (using `std::move`). Operators optionally take a `State&&`-parameter via the `updateState()` method and take ownership.

At construction of the hazard-affected algorithm objects, the dispatcher does not know how many tuples will be processed with each object. Therefore, the initialization cost must be $O(1)$, and any memory allocations must be lazily initialized to ensure that initialization costs are only incurred if the memory is used. However, when processing with the hazard-robust algorithm is deferred until after the hazard-affected algorithm pass is complete, the dispatcher knows the number of tuples that the hazard-robust algorithm will process. Therefore, the initialization cost can be $O(inputTuples)$.

Decreasing Robustness. As well as increasing the robustness of a hazard-adaptive operator, it must be possible to decrease the robustness to adapt to repeated changes in the data characteristics.

If a hazard-affected algorithm is being executed and generating few hazards the monitor can signal to the dispatcher that it would be beneficial to switch to a more hazard-affected algorithm by using a negative adjustment value for the `adjustRobustness()` call.

However, if the hazard-robust algorithm is being executed the design principle of micro-architectural efficiency means that the work executed is entirely independent of the key data characteristic. This is illustrated in Figure 1 where the performance of the predication-based implementation is independent of the predictability of the data. As a result, the performance counters provide no useful information to the monitor and the dispatcher cannot make an informed decision of when to switch to an alternative algorithm. This leaves two methods for switching from hazard-robust algorithms:

(1) For operators where higher-level order-invariant metrics (e.g., selectivity) can be determined at runtime these can be used to infer key data characteristics and therefore identify a subset of cases in which it is preferable to switch back to the hazard-affected algorithm (e.g., 0% selectivity implies full predictability).

(2) The dispatcher can choose to periodically perform probes (tests) using a hazard-affected algorithm which will switch optimally. These probes should run the minimum tuples necessary to obtain a statistically reliable value for the dominant event frequency.

All hazard-adaptive operators use **(2)**, with **(1)** also being used opportunistically when permitted by the operator's semantics.

4 HAZARD-ADAPTIVE OPERATORS

Creating hazard-adaptive operators is non-trivial, requiring the selection of appropriate foundational algorithms, identification of the dominant micro-architectural event(s), development of accurate heuristics for the monitor, and efficient switching between implementations by the dispatcher. We develop three such operators and discuss their implementation in the proposed framework.

The problem space for hazard-adaptive operators is broad, encompassing various operators and implementations that are specific to each database management system. To validate the concept of hazard-adaptive operators, we cover `SELECT`, `JOIN` and `GROUP`, collectively covering a substantial portion of this space.

4.1 Hazard-Adaptive SELECT Operator

Foundational Algorithms. A hazard-affected implementation of the `SELECT` operator is the standard branching algorithm defined as: `if(/*condition*/) output[j++] = input[i++];`. Work is minimized by only updating the output if the conditional predicate is satisfied. However, the use of a conditional can result in branch mispredictions (control hazards) leading to pipeline flushes [24].

To create an alternative branch-free implementation the technique of predication can be applied, always copying to the output: `output[j] = input[i++]; j += (/*condition*/);`. This makes the algorithm robust to control hazards at the cost of the extra work of updating the output before testing the conditional predicate.

For the branch-free algorithm, the number of instructions per tuple is constant and the performance is dependent on the selectivity of the data which determines the size of the output and consequently, additional reads and writes.

Monitoring Heuristics. The dominant micro-architectural event for the branching algorithm is branch mispredictions which occur when the branch predictor does not work effectively. Branch mispredictions dominate the performance since the resulting pipeline flushes cause costly pipeline bubbles in the instruction pipeline.

Branch mispredictions correlate with the (order-dependent) key data characteristic of predictability, rather than the higher-level (order-invariant) metric of selectivity. Figure 1 illustrates the importance of distinguishing these two metrics as varying the predictability of the input data at a fixed selectivity impacts the performance of the branching implementation due to branch mispredictions.

The monitoring heuristic is defined by a threshold value for the frequency of branch mispredictions. When this threshold is breached the monitor sends a signal to the dispatcher to increase the operator robustness, i.e., switch to the branch-free algorithm.

Dispatching Logic. Figure 3 shows how the `SELECT` operator processes an input composed of regions of high data locality (predictable) and low data locality (unpredictable). The dispatcher starts by processing tuples with the hazard-affected algorithm ❶ before

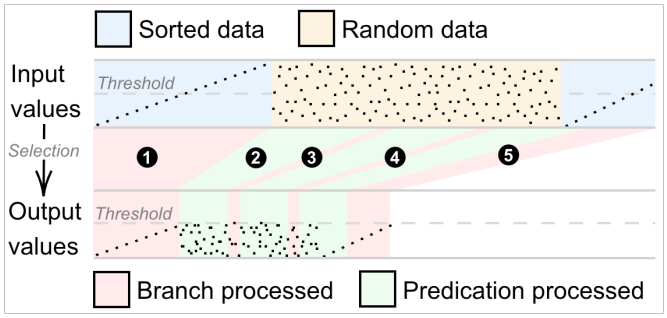


Figure 3: Hazard-adaptive processing of dataset with changing data locality using probing of hazard-affected algorithm

switching to the hazard-robust branch-free algorithm when it receives a signal from the monitor ❷. After processing a set number of tuples with the branch-free algorithm, the dispatcher runs a probing phase by switching back to the branching algorithm for a small micro-batch to determine whether the input has changed sufficiently for the branching algorithm to be superior (❸ to ❺).

As well as running probes, the dispatcher also contains logic for switching back to the branch-based algorithm based purely on the (order-invariant metric) selectivity. For example, at a very low or high selectivity, the hazard-affected algorithm is always superior.

4.2 Hazard-Adaptive GROUP Operator

Foundational Algorithms. A hazard-affected implementation of the `GROUP` operator is a hash-based aggregation. Tuples are inserted into a hash table and work is minimized by aggregating a payload immediately with the corresponding existing aggregated payload in the hash table. The algorithm returns an unordered array of unique keys with their corresponding aggregated payload. This algorithm is susceptible to capacity cache misses (data hazards). Capacity cache misses occur due to a random memory access pattern on a data structure that exceeds the cache size. This leads to "cache thrashing" due to frequent data swapping into and out of the cache.

In the implementation, we used Tessil's `robin_map` [34] which utilizes "Robin Hood" hashing [9] and outperformed other hash tables[4]. Note that we modify it to be lazily initialized ($O(1)$ complexity) in line with the requirements in Section 3.4.

An alternative hazard-robust algorithm must have less exposure to data stalls meaning a reduced cache footprint. One approach is to replace the random memory access pattern with a sequential access pattern. This requires a multi-pass algorithm which will allow for prefetching data and improve cache line utilization. Radix sort is a multi-pass algorithm that can create a sort-based implementation. Once the input is sorted a final pass is performed and adjacent tuples with matching keys are aggregated. However, this comes at the cost of unnecessarily sorting the output.

The fan-out, defined as $2^{radixBits}$, which determines the number of partitions (and therefore typically pages) written to during the data shuffling phase of radix sort, must be below that of the sTLB[5]

[4]Google `dense_map`, Abseil `flat_map`, Tessil `hopscotch_map`, Facebook `F14_map`.
[5]Shared TLB, also known as unified TLB (uTLB), or L2 TLB. Note that despite the name this is, like the L1 TLB, per-core.

capacity to ensure that sTLB store misses (a data hazard) do not occur and the algorithm is hazard-robust.

Monitoring Heuristics. The dominant micro-architectural event for the hash-based algorithm is last-level cache misses. Of all cache misses it is last-level cache misses that dominate performance.

Last-level cache misses correlate with the (order-dependent) key data characteristic of variability, rather than the higher-level (order-invariant) metric of cardinality. The difference between these two metrics is illustrated by inputs of equal cardinality and different clustering[6]. A clustered input means only a subset of the hash map will be repeatedly accessed at each point in the input. Therefore, clustering reduces the working set size (cache footprint), and a hash map larger than the cache can be used without cache thrashing.

Therefore the monitor heuristic is based on the frequency of last-level cache misses. However, a disadvantage is that a relatively large number of tuples is required to determine whether the working set size exceeds the last-level cache. Since the hash map is lazily initialized, with memory allocated with `calloc`[7], when pages are first updated they need to be allocated and initialized which increases the cost of these initial writes. Therefore the cost is non-linear with a disproportional cost for the initial tuples. This makes an efficient implementation of a hazard-adaptive `GROUP` operator challenging.

Identifying when to switch as early as possible is the only way to mitigate this cost. For the hash-based `GROUP` algorithm page faults can be used since the first time a page is touched it must be initialized (generating a page fault). The frequency of page faults gives a much quicker indication of the working set size vs. last-level cache misses when processing the first tuples. However, the page fault rate is transient. Therefore the heuristic must use a % of the input and be based on both the page fault frequency and the rate of decrease over a number of tuples. In other words, if the page fault rate is high and only decreases slowly, this means the working set size must be large and an increase in robustness is required.

Therefore the monitor not only requires a simple heuristic for last-level cache misses but also employs the monitoring of page faults to quickly identify when to switch in a subset of cases.

Dispatching Logic. For efficiency, the hazard-affected object reuses the hash map for all micro-batches processed. Additionally, the hazard-robust algorithm is deferred until after the hazard-affected algorithm has processed all the necessary micro-batches (of the three operators this out-of-order processing optimization is only required for `GROUP`). The sort-based algorithm processes the remaining micro-batches of the input in addition to the results collated in the hash map which forms the state of the hash-based implementation. This state is passed into the hazard-robust algorithm at construction using move semantics (Section 3.4). At construction, the additional radix sort buffer can be eagerly initialized since the dispatcher knows the total number of tuples that will be processed.

We rely on an oracle for cardinality estimates and apply an industry standard 2.5× over-allocation factor. The work of making a hash-based `GROUP` operator that is robust to cardinality estimates

[6]Defined as the similarity between adjacent values, e.g., clustering of 1,000 means that neighboring values would be ≤ 1,000 positions apart if the input array were sorted.
[7]When memory is allocated with `calloc`, the allocated memory all points to a single zeroed-out "copy-on-write" memory page.

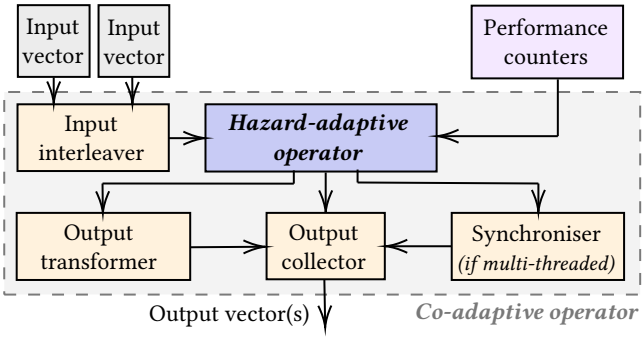


Figure 4: Co-adaptive operator architecture schematic

(analogous to `JOIN` [28]) is orthogonal to our work and can simply be incorporated by switching out the foundational algorithm.

Analogous to Figure 3 the dispatcher runs a probing phase of the hazard-affected algorithm. However, as the hazard-robust algorithm is deferred until after the hazard-affected algorithm the hazard-affected algorithm effectively jumps forward to run probes, leaving a block of input tuples for the hazard-robust algorithm.

5 CO-ADAPTIVITY

While effective for single-input operators, the hazard-adaptive architecture in Figure 2 is ill-suited for operators with multiple input tables such as `JOIN`, `PARTITION` (of multiple tables which we consider an operator), and `EXCEPT`. For example, consider a hazard-adaptive `INTERSECT` operator implemented with a hash-based approach and a sort-merge approach. When processing the tables sequentially starting with the hash-based approach, it is possible that the first table would produce few cache misses and be fully processed, but then the second table could generate many cache misses. At this point either a switch to a sort-merge approach would need to be made (throwing away all the work to create the hash table for the first table), or suffer the performance degradation from the cache misses. This disjoint processing of input tables fails to capture the data characteristics of both tables simultaneously.

To address this we define a new class of micro-adaptive operators: *co-adaptive* operators. In these operators, tables are considered together and co-processed, basing adaptivity on all tables.

5.1 Co-Adaptive Architecture

To create an architecture for co-adaptive operators we extend the hazard-adaptive operator architecture in Figure 2, resulting in Figure 4. We introduce an "input interleaver" that marks input tuples and passes them to the hazard-adaptive operator as if they were a single table. The hazard-adaptive operator processes micro-batches from each table in turn, making the monitor's adaptivity decisions based equally on all tables. The information denoting the table that tuples belong to is propagated through the operator to the "output collector" which collates the results. This process is shown in Figure 5 for the `PARTITION` operator (discussed in Section 5.2).

For co-adaptive operators where the foundational algorithms have different (but valid) outputs (e.g., partitioning using different radix bits values) there is the challenge of ensuring the structure of results in the output collector is consistent with the active operator.

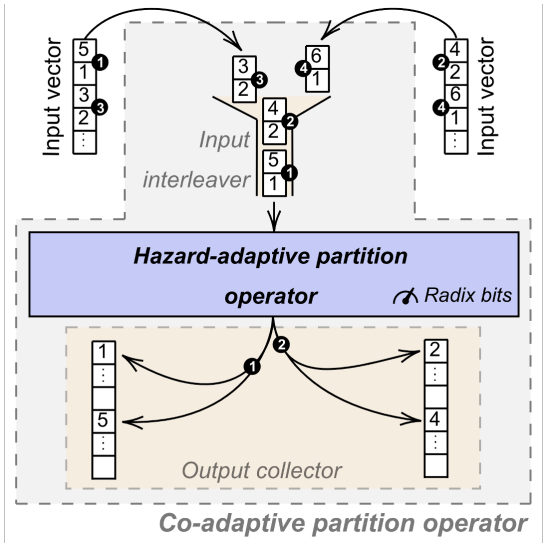


Figure 5: Co-adaptive partition operator

To address this we include an "output transformer" component that receives a signal from the monitor when the robustness of the operator is adjusted. The output transformer ensures consistency between the active operator and the results in the output collector.

To maintain synchronization of threads when multi-threaded we include a "synchronizer" that ensures all threads are at the same point in the adaptivity space and the output format is consistent.

Returning to the `INTERSECT` operator example, using the co-adaptive operator architecture and hashing parts of each table in turn (generating an output when there is a match) a switch can be made to the sort-merge implementation without losing progress.

5.2 Co-Adaptive `JOIN` (`PARTITION`) Operator

A hash-join begins to suffer last-level cache misses when the hash table from the build side (the smaller table) does not fit in the last-level cache. Such a join can be made robust by partitioning the two input tables and then joining these partitions (which now fit in the cache) independently and unioning the result (known as a radix partitioned hash join [22]). Therefore, we focus on the `PARTITION` operation as this is the critical phase to make these large table joins robust. Furthermore, a highly efficient `PARTITION` operator has a range of applications as it forms the basis for many operators [40]. By limiting the `PARTITION` operator to `JOIN` we make this a co-adaptive operator since a `JOIN` has two input tables. Therefore we use the co-adaptive architecture in Figure 4.

Foundational Algorithms. To minimize the work required for radix partitioning a large fan-out, defined as $2^{radixBits}$, can be used to partition data with a single partitioning pass. However, a large fan-out is susceptible to data hazards in the form of sTLB store misses during the data shuffling phase of partitioning.

During the data shuffling phase values are moved to their respective partition. Each partition typically resides on a separate page, requiring a TLB entry [3]. If the fan-out exceeds the sTLB capacity and a random partition access pattern is required this causes sTLB store misses and subsequent data stalls. A high sTLB store miss rate is known as "TLB thrashing" [4] (analogous to cache thrashing).

The TLB footprint of the hazard-affected `PARTITION` algorithm is reduced by converting it to a multi-pass algorithm: using a smaller number of radix bits, and recursively partitioning. By using a TLB footprint (fan-out) smaller than the sTLB capacity the algorithm will not suffer sTLB misses and is hazard-robust.

As the algorithm is hazard-robust the performance per tuple is broadly independent of the data. However, unlike for `SELECT` there is a spectrum of hazard-affected algorithms, with different fan-outs between the fully robust and single-pass algorithm. Utilizing this spectrum of algorithms maximizes the range of inputs that the hazard-adaptive operator can process optimally.

Monitoring Heuristics. The dominant micro-architectural event for high fan-out radix partitioning is last-level sTLB store misses. Therefore, the system monitors the TLB store miss rate to adjust the hazard-adaptive operator's robustness. As the TLB store miss rate for a specific fan-out depends on the (order-dependent) key data characteristic of "clusteredness" (which determines the number of "cache-hot" partitions) it is a good proxy metric for clusteredness.

Dispatching Logic. The optimal number of radix bits to partition an input depends on the clusteredness of the data, with greater clusteredness meaning that a larger number of radix bits can be used without generating excessive data hazards. As for all hazard-adaptive operators, the dispatcher starts executing micro-batches on the most hazard-affected algorithm, i.e., the highest radix bits value. Since this is a co-adaptive operator the input interleaver will alternate between the two tables and the monitor will send signals to the dispatcher based on the hazards generated by both tables.

When the dispatcher receives a signal to increase robustness it decrements the number of radix bits and signals the output transformer to merge the necessary partitions in both output tables in the output collector to match the partitions associated with the new radix bits value. The associated histogram values of both tables are also updated. The synchronizer is also called in a multi-threaded implementation to ensure this consistency across threads.

This approach limits the operator's use to homogeneous data (see Section 7.3) since the radix bits value can only be decreased. The radix bits value could be reset for recursive partitioning to deal with heterogeneous data (since the order of the data is retained within partitions) or retain the radix bits value if the data is expected to be homogeneous. Either choice could leave performance on the table and the optimization of this decision is left for future work.

Finally, if the minimum radix bits value is reached the entire remaining input is processed since no more adaptivity is possible.

6 SYSTEM INTEGRATION

To test the presented concepts we need to integrate them into a system. To achieve this we used the BOSS system which required some technical design decisions that we outline in this section.

6.1 BOSS System

To use hazard-adaptive operators without developing a new system we build on an existing execution engine. Composable data management systems (DMS) are designed for easy extensibility and allow

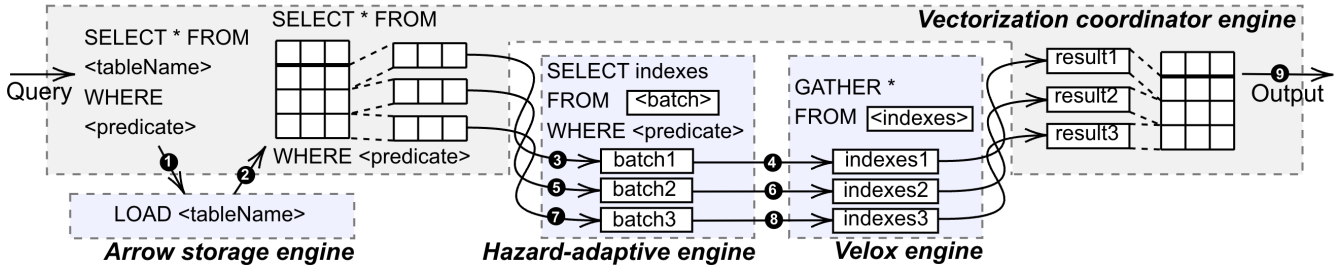


Figure 6: Cross-engine vectorization in BOSS system pipeline

us to build just the operators amenable to adaptivity. Therefore we use "BOSS" [25], our next-generation composable DMS.

Velox and Apache Arrow. The means of extensibility in BOSS is an engine. Therefore we combine the operators into a hazard-adaptive engine and integrate this with Apache Arrow[8] (for storage) and Velox[9] (for execution) with minimal overhead. With this system, the engines are loosely coupled and we can easily augment or replace functionality in Velox with that of the hazard-adaptive engine with virtually no changes to Velox.

Non-Adaptive Engine. We found Velox's `GATHER` operator to have some overhead and therefore, rather than changing the Velox kernel directly, we created a standalone "non-adaptive engine" with a non-adaptive `GATHER` operator and a `JOIN` operator (accepting indexed tuples from the upstream `PARTITION` operator). However, this is an implementation detail and, for the sake of simplicity, we will discuss the Velox and non-adaptive engines as if they were one.

6.2 Cross-Engine Vectorization

Query evaluations start with the Apache Arrow storage engine loading tables. Following this the hazard-adaptive engine is used to perform partial query evaluation on the operators (or parts of operators) that can benefit from adaptivity, e.g., index creation step for `SELECT` (Section 4.1). The remaining stages of these operators (e.g., gather following index generation in `SELECT`) and other non-evaluated operators are left for evaluation in the Velox engine.

Velox is a vectorized execution engine. Therefore to achieve competitive performance the hazard-adaptive engine is also vectorized (using ~1MB vectors and in the style of X100 [41] or DuckDB [31]). However, BOSS does not impose an execution mode (i.e., it has no notion of vectorization) and would materialize all intermediate results. Therefore we create a "Vectorization coordinator engine". This engine breaks down the query into pipelineable chunks and dispatches the vectors to the relevant engines. This process of "cross-engine vectorization" and the system architecture is shown in Figure 6 with the numbers representing the order of execution of engines. Following loading the tables by the Arrow storage engine ❶, the tables are split into vectors ❷. Each vector is passed through the pipeline of execution engines, the "Hazard-adaptive engine" and the "Velox engine", as shown by ❸ to ❽, before being unioned by the vectorized coordinator engine and returned ❾. Should there be any pipeline breakers, these must be batch-evaluated and fully materialized since they cannot be vectorized. For example, `GROUP` and `PARTITION` are batch-evaluated in the hazard-adaptive engine.

6.3 Parallelization

We leverage vectorization to perform parallelization across vectors since they can be processed independently. The individual results are collected and unioned within the vectorization coordinator engine. This makes parallelization straightforward, using thread local storage to store operator objects for reuse within each worker thread. However, the operators are executed independently and consequently are unaware of the DOP and thus the correct machine constant value to use within a heuristic. Therefore the vectorization coordinator engine also passes the DOP to the hazard-adaptive engine (embedded within the BOSS expression alongside the query).

This approach for parallelization of Velox could be considered a downside as we are reimplementing simple parallelization, however, we leave the task of utilizing in-engine parallelization when using cross-engine vectorization for future work.

6.4 Metaparameters and Trade-Offs

A key metric of adaptive algorithms is the maximum rate of change of the input data that the algorithm can adequately adapt to. This metaparameter, which we refer to as the "adaptive period", is a necessary condition of adaptive algorithms and determines many implementation design choices. Naturally, the required adaptive period depends on the input data. Thus implementing an adaptive algorithm with a static adaptive period to work effectively across a range of datasets requires considering several trade-offs.

All of the switching mechanisms and tests discussed in Sections 3.3 and 3.4 incur an overhead. This overhead is amortized over the tuples processed between tests. Therefore the frequency of reading performance counter values, comparing high-level characteristics to tipping point thresholds, or performing probes of the hazard-affected algorithm is always a trade-off of adaptive responsiveness (smaller adaptive period) vs. overhead. Consequently, the frequency of these tests should be set to achieve a satisfactory adaptive period (responsiveness) without generating too much overhead.

6.5 Performance Counters and Heuristics

Performance Counters. Reading performance counters requires the necessary permissions to access the performance monitoring unit

[8] https://github.com/apache/arrow
[9] https://github.com/facebookincubator/velox

(PMU)[10] [29]. This can also be done from within a container or VM [14]. Furthermore, since performance counters exist on a per-core basis a machine will always have sufficient counters since the number of operators running never exceeds one per core.

Simplifying Heuristics. The performance of hazard-robust algorithms is not entirely constant, this is primarily due to two factors. Firstly, where the output size is variable it naturally affects performance. Secondly, multi-pass algorithms include step changes in performance depending on the number of passes. Therefore heuristics should account for these factors by adjusting the required frequency of events necessary for the monitor to observe before it sends a signal to the dispatcher to increase hazard robustness. For example, adjusting it based on the selectivity/cardinality of the data, or scaling it by the number of passes required for multi-pass algorithms (where this differs from the value used to create the heuristic). However, in practice, we found that a simple fixed value closely followed the optimal static implementation. This simplification is feasible due to the consistency of the tipping point across various inputs and the only minor impact of different output sizes.

Calibrating Machine Constants. A relatively large number of machine constants are required (~100 for our experiments). However, since these are truly constant they can be calculated during installation. An automated experimental calibration process using binary search determines the switch-over point between foundational algorithms. The corresponding event frequency at this point is then measured. During this calibration process a low interference environment is preferable to capture the maximum performance.

7 EVALUATION

We limit our evaluation to Intel processors. However, our approach and operators are equally applicable to AMD and ARM architectures given their similar handling of branch prediction, cache management, use of TLBs, etc., and analogous hardware performance counters which can also be accessed using PAPI [29].

7.1 Experimental Setup

Hardware. All experiments are performed on a server with two Intel Xeon Silver 4114 2.20 GHz CPUs, each with 10 physical cores, a 14 MB LLC cache, and 196 GB of memory. Each core has 1,536 TLB entries for 4KB pages (L2 TLB) and 4 entries for 1GB pages (L1 TLB). We limit multi-threaded experiments to a single NUMA node to exclude NUMA effects. We use Ubuntu 18.04 with Linux kernel 4.15.0-209 running in a Docker container and compile all code with Clang version 14 using flags `-O3 -march=native -std=c++20`.

Systems and Setup. To implement our system, we used BOSS v0.3, Apache Arrow v6.0.1, and Velox v0.0.1a0 (git rev. 164903523e39). Our implementation required two (minor) modifications to Velox's code: exposing the Task's internal child pools for taking ownership of the output data and adding support for arithmetic operators of mixed data types. When combined with our accelerator we merge Velox's output batches to achieve the desired size (i.e., 300,000 rows) in our code and leave Velox's parameters decoupled and as default.

[10]The Linux paranoid level (`/proc/sys/kernel/perf_event_paranoid`) must be ≤ 1

We measure execution time using Google Benchmark, evaluating each query with a single-timed run following three warm-up runs.

Workloads. For macro-benchmarks, we use TPC-H [35] at scale factor (SF) 100 (i.e., 100GB). Following the established practice [20], we evaluate five queries that capture the benchmark's choke points [6].

For handling string columns with low cardinality, BOSS' storage engine implements compressed string dictionaries and treats these columns as integer types for the rest of the query processing. Due to the lack of support for non-dictionary compressed string columns in BOSS yet, Q9 is modified to filter the part table on p_retailprice rather than p_name and Q18 to group by o_orderkey only. However, the cardinalities are preserved. Q1, Q3, and Q6 are unchanged.

Query plans are hand-written using fixed heuristics: to decide the join order for each query, we iteratively pick the two smallest among all the pairs of relations that can be joined next and always use the smaller of the two on the build side.

Micro-benchmarks use 64-bit ints for both keys and payloads. All benchmarks were also repeated with 32-bit ints with no differences.

7.2 Performance with TPC-H Benchmark

We first evaluate single-threaded performance on TPC-H. To test the robustness (i.e. maximizing worst-case performance) of the hazard-adaptive operators we select dimensions to sort the TPC-H tables on and then progressively randomize these tables to create pathological cases for both hazard-affected and hazard-robust algorithms.

We also examine performance on the default TPC-H tables and, despite the benefits of hazard-adaptive operators being orthogonal to those from parallelism, we include multi-threaded performance.

Modified TPC-H Tables. To modify TPC-H tables we sort at least one table on at least one column to maximize the performance of the dominant operators in the associated query. We believe that the dimensions that we have selected generate near the minimum frequency of hazards possible, however, we cannot prove this. Therefore, another set of dimensions may exist that creates a more pathological test case, and consequently, the maximum upside from the hazard-adaptive operators could be higher.

We include two accelerator results, one using the Gather operator in Velox (slow Gather) which obscures some hazards due to additional overhead, and one using our own Gather operator (fast Gather) leveraging our composable architecture.

Q1, Q3, and Q6 include large selections accounting for a substantial proportion of the query cost. Therefore, we sort the large tables (lineitem and orders) by the columns used in the selection predicates (l_shipdate for Q1; l_shipdate and o_orderdate for Q3; and multi-level sorting by l_quantity, l_discount, and l_shipdate for Q6). Figure 7 shows for these queries the performance of Velox degrades with increasing randomness due to the associated increase in branch misprediction frequency. This degradation is worse for Q3 relative to Q1 due to the selectivity being closer to 50% in Q3 (48% and 54%) versus Q1 (98%) which increases the exposure to branch mispredictions. Q6 shows a further step in degradation due to the first three predicate selectivities of 46%, 55%, 50%, and the overall selectivity of only 1.9% making the remaining query comparatively cheap to evaluate. In contrast, the hazard-adaptive accelerator (with fast Gather) provides robust performance from sorted to random

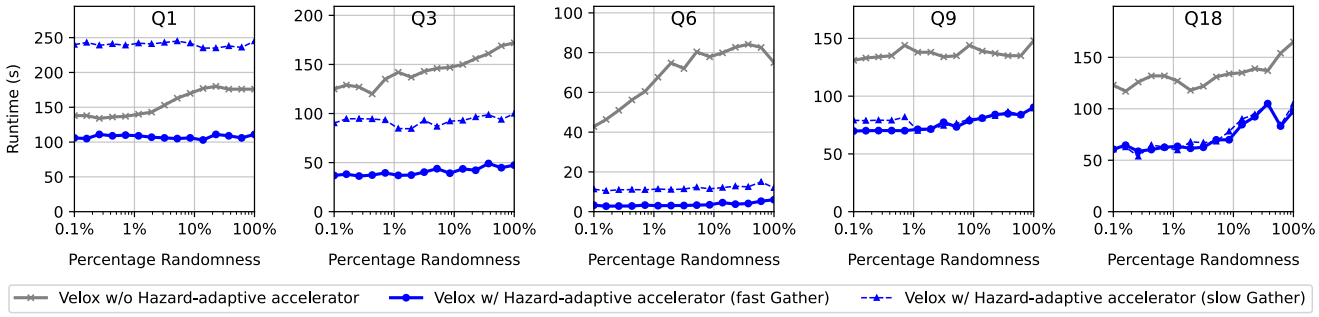


Figure 7: Single-threaded runtime for TPC-H on five representation queries at SF100, varying sortedness of key tables

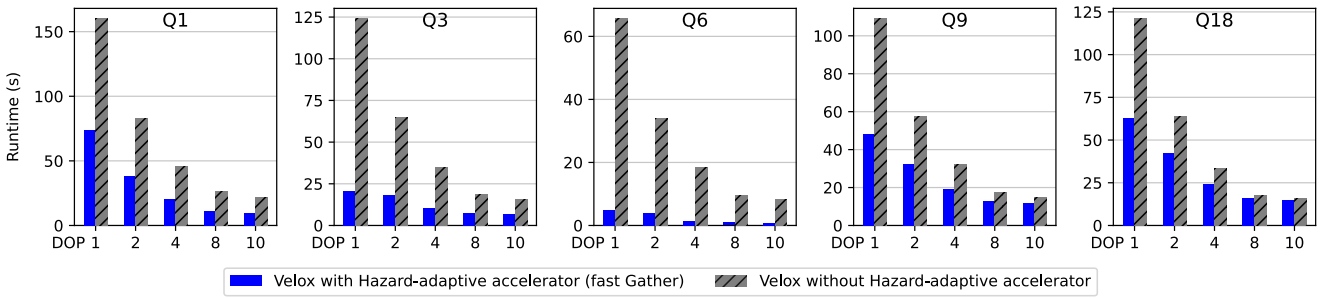


Figure 8: Runtime for TPC-H on five representation queries at SF 100, varying degrees of parallelism (DOP)

and performance benefits up to a peak of ~20×. Since selections are important in these queries, the performance of the Gather phase is key and the overhead of the slow Gather is significant.

Joins dominate the cost for Q9. The join with the orders table (150M rows) requires the largest build side (~30M rows) and therefore has the potential to generate the most hazards in a hash-join. Consequently, we sort lineitem and orders by orderKey. However, since the key on the probe side (orders) is the primary key (orderKey) it is unique and, as a result, the sortedness has minimal impact on reducing hazards during the probe phase (a lower cardinality would mean repeated keys during the probe phase and a greater cache hit frequency). We see this on Q9 in Figure 7 with the performance of Velox not degrading with increased randomness. This is because a high frequency of cache misses is observed regardless of the degree of randomness. However, this is still a pathological case for the co-adaptive `PARTITION` operator with the TLB miss rate determined by the sortedness of the key. By adapting to locality within the two keys the co-adaptive partition operator delivers high performance and robustness across the problem space.

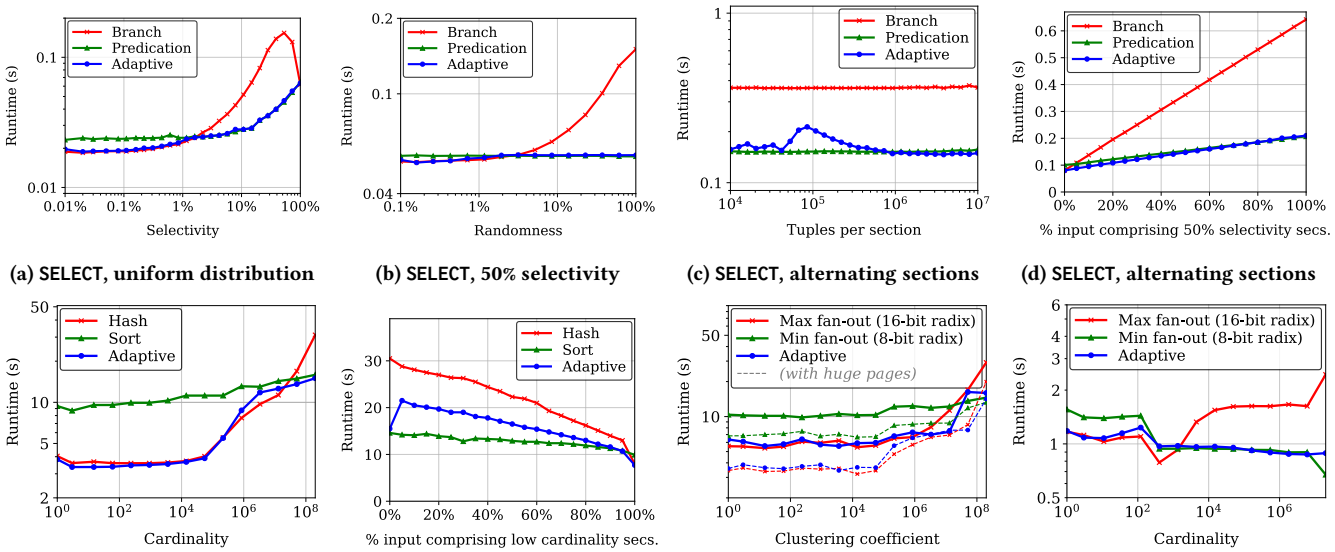
Q18 is dominated by the cost of the high cardinality (150M) aggregation of the lineitem table (600M rows) on the orderKey and the large join between the customers (15M rows) and orders (150M rows) tables. Therefore we sort lineitem by orderKey, and orders and customers by custKey. Velox does show some performance degradation, however, this is minimized by significant prefetching during hash probing and overhead due to Velox's small internal batch size of 1024. When removing prefetching and increasing the internal batch size (including joins) to minimize overhead on the test machine we observed a performance degradation of at least 2×. The hazard-adaptive accelerator improves performance, switching to sort-based aggregation at ~50% randomness to prevent further performance degradation due to excessive cache misses.

Default TPC-H Tables. Figure 8 shows single and multi-threaded performance on the default TPC-H tables. Even on unmodified data, the hazard-adaptive accelerator provides a single-threaded performance improvement of ~2-20× across the queries by correcting poor decisions of the optimizer. This is primarily due to the unpredictable data in the selections and the sortedness of the data in the large joins/aggregations that the adaptive operators can leverage.

Velox demonstrates good scalability with increased DOP, achieving relatively close to linear speedup. The hazard-adaptive accelerator also achieves good performance improvement with increased DOP, becoming memory-bound in Q6. Across all five queries and DOP values, the hazard-adaptive accelerator provides Velox with a performance benefit. However, this benefit is reduced for Q9 and Q18, particularly at high DOP. This is due to the large joins which require the hazard-adaptive `PARTITION` operator. Efficient parallelization of the first radix partitioning pass within this operator is a significant engineering challenge due to the extensive merging and information sharing required between threads.

7.3 Micro-Benchmarks

SELECT. Figure 9 (a)-(d) contain the micro-benchmarks for `SELECT`. Figures (a) and (b) show operator performance on homogeneous data, i.e., datasets where the defining data characteristics remain



(a) SELECT, uniform distribution **(b) SELECT, 50% selectivity** **(c) SELECT, alternating sections** **(d) SELECT, alternating sections**

(e) GROUP, uniform distribution **(f) GROUP, alternating sections** **(g) PARTITION, 200M cardinality** **(h) PARTITION, uniform distribution**

Figure 9: Micro-benchmarks: Individual hazard-adaptive operator performance versus associated foundational algorithms

consistent throughout the dataset. Figure (a) varies the higher-level order-invariant data characteristic of selectivity, whereas Figure (b) varies the order-dependent characteristic of randomness (i.e., % of randomly shuffled data in a sorted input) at a fixed selectivity of 50%. Both figures show that the hazard-adaptive operator tracks the optimal static implementation with negligible overhead.

Figures (c) and (d) show operator performance on heterogeneous data, i.e., datasets where the data characteristics vary throughout the dataset. Both figures comprise sections of high selectivity (50%) and low selectivity (0%). Figure (c) varies the number of tuples in each section, with the total number of tuples fixed at 100M. The performance of the two foundational algorithms is independent of the number of tuples in each section because the overall average selectivity and predictability within the input remain constant, i.e., only the frequency of switching selectivity is varied. We observe that when sections contain a large number of tuples the performance of the hazard-adaptive operator is superior to both of the foundational algorithms. This is because the hazard-adaptive operator selects the optimal foundational algorithm to process each section thereby achieving better performance than any static implementation.

Conversely, when there are a small number of tuples in each section the adaptive algorithm cannot work effectively as the input is changing more frequently than the adaptive period of the algorithm (average of 50,000 tuples for the implementation of SELECT). For these inputs, the adaptive algorithm cannot adapt fast enough and will predominantly run the hazard-robust algorithm.

Between these two extremes, there is a decrease in performance. This rise occurs when the length of sections is on the order of the adaptive period causing the algorithm to often select the incorrect implementation for the next micro-batch. This, again, occurs because the input is varying too fast for the adaptive algorithm to respond to, leading to many tuples being processed using a sub-optimal implementation. However, by adding some randomness to the number of tuples processed between counter readings the chance of the dispatcher "flip-flopping" between incorrect implementations is minimized. More sophisticated strategies such as dynamically adjusting the adaptive period to the rate of change of data locality are possible and are an avenue of future work.

In Figure (d) we fix the length of input sections (at a length that the hazard-adaptive operator is effective) and instead vary the number of 0% selectivity sections relative to the number of 50% selectivity sections. Therefore, when 100% of the input comprises 0% selectivity sections the input corresponds to a uniformly distributed input with 50% selectivity. At the other extreme, the input corresponds to a 0% selectivity input. Between these two extremes, the performance of all algorithms varies linearly as the relative number of sections is altered. The adaptive algorithm benefits from being able to optimally process the individual sections of the input and provides superior performance to both foundational algorithms.

GROUP. Figures (e) and (f) contain the micro-benchmarks for GROUP. Figure (e) varies the cardinality of the input with uniform data (i.e., homogeneous data) and shows that the hazard-adaptive operator tracks the optimal static implementation with minimal overhead due to lazily initializing the hash map and repeatedly quickly identifying the optimal static implementation using the page fault rate.

Figure (f) shows operator performance on heterogeneous data, with sections varying between high and low cardinality. The hazard-adaptive operator does track the optimal static implementation (this can be observed clearly at the two extreme values), however, there is a greater degree of overhead (observable as the "Adaptive" line above rather than below the "Sort" line). The tipping points to switch to the sort-based algorithm are identified by the last-level cache miss rate (rather than the page fault miss rate which can only identify the tipping point when the hash table is empty, i.e., homogeneous datasets) which requires processing more tuples to

get a converged and statistically accurate hazard rate. This increases the overhead due to an increased number of cache misses and, more importantly, page faults which require initialization. This overhead is a key challenge of a hazard-adaptive GROUP operator which can optimally process both homogeneous and heterogeneous datasets.

JOIN (PARTITION). Figures (g) and (h) contain the micro-benchmarks for PARTITION, with both benchmarks on homogeneous data since this co-adaptive operator is not suited to heterogeneous data as described in Section 5.2. Figure (g) varies the order-dependent key data characteristics of clustering, whereas Figure (h) varies the order-invariant data characteristic of cardinality.

In both figures, the hazard-adaptive operator closely tracks either the "Max fan-out" or "Min fan-out" line with only minor overhead. Between a clustering coefficient[11] of $\sim 10^6 - 10^7$ on Figure (g) the performance of the hazard-adaptive operator exceeds that of both static implementations. This is due to the hazard-adaptive PARTITION operator being comprised of a spectrum of hazard-affected algorithms (one for each radix bits value between the bounding values). Therefore, for some clustering values, the input is optimally processed using one of these intermediate radix bit values. When we compared the performance of the hazard-adaptive operator to the optimal radix bit value (i.e., the optimal static implementation with radix bit values between 8-16), we observed that it closely follows the optimal implementation. This demonstrates the benefit of using multiple hazard-affected foundational algorithms. We found this phenomenon to be far more pronounced on other machines.

Figure (g) also includes the results when forcing the OS to use huge pages, illustrated by the dashed lines. This machine contains 4 entries for huge pages, all in the L1 TLB. This greater use of the L1 TLB vs. L2 TLB means that all the algorithms run faster, and the point at which TLB thrashing occurs is delayed due to the large working set size associated with the TLB capacity. However, at the point of TLB thrashing the process of adaptivity works the same as for normal pages, observed at the far right-hand side of the figure.

In Figure (h) we use a fixed maximum value of 20M with values spread between 1 and this maximum value. For low cardinality datasets, due to the large number of repeated small values, an additional pass is required by the minimum fan-out algorithm to partition the data (data is partitioned starting from the most significant bits). This results in a "step-shaped" improvement in performance at a cardinality of $\sim 10^2$ when a single partitioning pass becomes sufficient. It is also at this point that the maximum fan-out algorithm begins to suffer from TLB thrashing.

Parallelization. Where the tipping point depends on a shared resource, parallelization changes the contention for the resource and thus moves the tipping point. For SELECT, predication requires more memory bandwidth, so the tipping point shifts to favor the branch-based implementation up to a higher level of unpredictability with increased DOP. Similarly for GROUP, increased contention for the shared cache moves the tipping point in favor of the sort-based implementation to lower levels of variability. On the other hand, selecting the foundational algorithm for PARTITION does not depend on a shared resource (since each core has its own TLB), and therefore is independent of parallelization.

[11] Defined as the similarity between adjacent values, e.g., clustering of 1,000 means that neighboring values would be $\leq$ 1,000 positions apart if the input array were sorted.

8 RELATED WORK

Traditionally, data characteristics are captured in histograms [10]. However, histograms do not capture aspects like auto-correlation. Approaches to determine order-dependent metrics before execution are complicated and brittle [1, 5, 16]. Our approach removes the need for analysis before execution, instead using micro-adaptivity to select the optimal operator implementation. Micro-adaptivity was introduced by Răducanu et al. [33]. They propose to create several operator implementations by applying techniques like loop fission or unrolling and using different compilers. They propose a multi-armed bandit (MAB) reinforcement learning algorithm [2, 8] to test operator implementations during an "exploration" phase and select the optimal implementation during an "exploitation" phase. Kaftan et al. expand this approach [19] and highlight the challenge of quickly and accurately selecting the optimal operator implementation (at least ~15% [33] and scaling with $O(implementations)$ [19]).

Grizzly [17], a stream processing engine, uses counters for two "adaptive optimizations". Fine-grained code instrumentation identifies optimizations and counters trigger re-optimizations in two areas. Firstly, identifying changes in predicate selectivity via branch mispredictions, triggering a profiling phase that incurs a ~50% drop in throughput. Secondly, determining the use of a shared versus independent hash map in a multi-threaded implementation using simultaneous exclusive accesses to cache lines to detect data skew.

Zeuch et al. [39] use counters without code instrumentation to drive progressive optimization [23] at the overall query level by triggering a re-optimization process based on runtime comparisons of counter readings against cost models [39]. However, using counters was only explored to measure selection output cardinalities to optimize the order of selection predicates of a single table. Similarly to Zeuch et al., we aim to maximize the potential of counters, in our case applied to the paradigm of micro-adaptivity. We also make our approach widely applicable by creating a generalized architecture.

9 CONCLUSION

Micro-architectural hazards are a key challenge for in-memory data processing systems. However, their prevalence depends on patterns within the data, making static query optimization ill-suited for this problem. We propose "white-box" micro-adaptive operators that continuously monitor micro-architectural hazards and advise switches to alternative implementations. We present architectures for hazard-adaptive and co-adaptive operators, implement key relational operators, and based on a next-generation composable database system, create an accelerator engine for Velox.

We demonstrate that white-box micro-adaptivity accelerates Velox by up to 20× on TPC-H benchmarks and is robust to varying degrees of data locality, including pathological cases. Micro-benchmarks confirm that the hazard-adaptive operators match the optimal static operator performance with minimal overhead.

The long-term objective of this line of research is fully dynamic optimization of operators and even full query plans. For this paper, we focused on OLAP but see further opportunities in transaction processing (e.g. processing data if cache resident).

REFERENCES

[1] Miklós Ajtai, T. S. Jayram, Ravi Kumar, and D. Sivakumar. 2002. Approximate Counting of Inversions in a Data Stream. In *Proceedings of the Thiry-Fourth Annual ACM Symposium on Theory of Computing* (Montreal, Quebec, Canada) *(STOC '02)*. Association for Computing Machinery, New York, NY, USA, 370–379. https://doi.org/10.1145/509907.509964

[2] Peter Auer, Nicolo Cesa-Bianchi, and Paul Fischer. 2002. Finite-time analysis of the multiarmed bandit problem. *Machine learning* 47 (2002), 235–256.

[3] Cagri Balkesen, Jens Teubner, Gustavo Alonso, and M. Tamer Özsu. 2013. Main-memory hash joins on multi-core CPUs: Tuning to the underlying hardware. In *2013 IEEE 29th International Conference on Data Engineering (ICDE)*. 362–373. https://doi.org/10.1109/ICDE.2013.6544839

[4] Maximilian Bandle, Jana Giceva, and Thomas Neumann. 2021. To Partition, or Not to Partition, That is the Join Question in a Real System. *Proceedings of the 2021 International Conference on Management of Data* (2021). https://api.semanticscholar.org/CorpusID:231978821

[5] Sagi Ben-Moshe, Yaron Kanza, Eldar Fischer, Arie Matsliah, Mani Fischer, and Carl Staelin. 2011. Detecting and Exploiting Near-Sortedness for Efficient Relational Query Evaluation. In *Proceedings of the 14th International Conference on Database Theory* (Uppsala, Sweden) *(ICDT '11)*. Association for Computing Machinery, New York, NY, USA, 256–267. https://doi.org/10.1145/1938551.1938584

[6] Peter Boncz, Thomas Neumann, and Orri Erling. 2013. TPC-H Analyzed: Hidden Messages and Lessons Learned from an Influential Benchmark. In *Revised Selected Papers of the 5th TPC Technology Conference on Performance Characterization and Benchmarking - Volume 8391*. Springer-Verlag, Berlin, Heidelberg, 61–76. https://doi.org/10.1007/978-3-319-04936-6_5

[7] Sebastian Breß, Bastian Köcher, Henning Funke, Steffen Zeuch, Tilmann Rabl, and Volker Markl. 2018. Generating custom code for efficient query execution on heterogeneous processors. *The VLDB Journal* 27 (2018), 797–822.

[8] Sébastien Bubeck, Nicolo Cesa-Bianchi, et al. 2012. Regret analysis of stochastic and nonstochastic multi-armed bandit problems. *Foundations and Trends® in Machine Learning* 5, 1 (2012), 1–122.

[9] Pedro Celis, Per-Ake Larson, and J. Ian Munro. 1985. Robin hood hashing. In *26th Annual Symposium on Foundations of Computer Science (sfcs 1985)*. 281–288. https://doi.org/10.1109/SFCS.1985.48

[10] Surajit Chaudhuri, Rajeev Motwani, and Vivek Narasayya. 1998. Random Sampling for Histogram Construction: How Much is Enough?. In *Proceedings of the 1998 ACM SIGMOD International Conference on Management of Data* (Seattle, Washington, USA) *(SIGMOD '98)*. Association for Computing Machinery, New York, NY, USA, 436–447. https://doi.org/10.1145/276304.276343

[11] Intel Corporation. 2016. *Intel 64 and IA-32 Architectures Optimization Reference Manual*. Intel Corporation.

[12] Andrew Crotty, Alex Galakatos, and Tim Kraska. 2020. Getting Swole: Generating Access-Aware Code with Predicate Pullups. In *2020 IEEE 36th International Conference on Data Engineering (ICDE)*. 1273–1284. https://doi.org/10.1109/ICDE48307.2020.00114

[13] Ulrich Drepper. 2007. What every programmer should know about memory. *Red Hat, Inc* 11, 2007 (2007), 2007.

[14] Jiaqing Du, Nipun Sehrawat, and Willy Zwaenepoel. 2011. Performance profiling of virtual machines. In *Proceedings of the 7th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments* (Newport Beach, California, USA) *(VEE '11)*. Association for Computing Machinery, New York, NY, USA, 3–14. https://doi.org/10.1145/1952682.1952686

[15] Stéphane Eranian. 2008. What Can Performance Counters Do for Memory Subsystem Analysis?. In *Proceedings of the 2008 ACM SIGPLAN Workshop on Memory Systems Performance and Correctness: Held in Conjunction with the Thirteenth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '08)* (Seattle, Washington) *(MSPC '08)*. Association for Computing Machinery, New York, NY, USA, 26–30. https://doi.org/10.1145/1353522.1353531

[16] Parikshit Gopalan, T.s Jayram, Robert Krauthgamer, and Ravi Kumar. 2007. Estimating the sortedness of a data stream. 318–327. https://doi.org/10.1145/1283383.1283417

[17] Philipp M. Grulich, Breß Sebastian, Steffen Zeuch, Jonas Traub, Janis von Bleichert, Zongxiong Chen, Tilmann Rabl, and Volker Markl. 2020. Grizzly: Efficient Stream Processing Through Adaptive Query Compilation. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data* (Portland, OR, USA) *(SIGMOD '20)*. Association for Computing Machinery, New York, NY, USA, 2487–2503. https://doi.org/10.1145/3318464.3389739

[18] John L. Hennessy and David A. Patterson. 2011. *Computer Architecture, Fifth Edition: A Quantitative Approach* (5th ed.). Morgan Kaufmann Publishers Inc., San Francisco, CA, USA.

[19] Tomer Kaftan, Magdalena Balazinska, Alvin Cheung, and Johannes Gehrke. 2018. Cuttlefish: A Lightweight Primitive for Adaptive Query Processing. arXiv:1802.09180 [cs.DB]

[20] Timo Kersten, Viktor Leis, Alfons Kemper, Thomas Neumann, Andrew Pavlo, and Peter Boncz. 2018. Everything you always wanted to know about compiled and vectorized queries but were afraid to ask. *Proc. VLDB Endow.* 11, 13 (sep 2018), 2209–2222. https://doi.org/10.14778/3275366.3284966

[21] Corey Malone, Mohamed Zahran, and Ramesh Karri. 2011. Are Hardware Performance Counters a Cost Effective Way for Integrity Checking of Programs. In *Proceedings of the Sixth ACM Workshop on Scalable Trusted Computing* (Chicago, Illinois, USA) *(STC '11)*. Association for Computing Machinery, New York, NY, USA, 71–76. https://doi.org/10.1145/2046582.2046596

[22] Stefan Manegold, Peter Boncz, and Martin Kersten. 2002. Optimizing Main-Memory Join on Modern Hardware. *IEEE Trans. on Knowl. and Data Eng.* 14, 4 (jul 2002), 709–730. https://doi.org/10.1109/TKDE.2002.1019210

[23] Volker Markl, Vijayshankar Raman, David Simmen, Guy Lohman, Hamid Pirahesh, and Miso Cilimdzic. 2004. Robust Query Processing through Progressive Optimization. In *Proceedings of the 2004 ACM SIGMOD International Conference on Management of Data* (Paris, France) *(SIGMOD '04)*. Association for Computing Machinery, New York, NY, USA, 659–670. https://doi.org/10.1145/1007568.1007642

[24] S. McFarling and J. Hennesey. 1986. Reducing the Cost of Branches. In *Proceedings of the 13th Annual International Symposium on Computer Architecture* (Tokyo, Japan) *(ISCA '86)*. IEEE Computer Society Press, Washington, DC, USA, 396–403.

[25] Hubert Mohr-Daurat, Xuan Sun, and Holger Pirk. 2024. BOSS - An Architecture for Database Kernel Composition. *Proc. VLDB Endow.* 17, 4 (mar 2024), 877–890. https://doi.org/10.14778/3636218.3636239

[26] Subrata Naskar and J. P. Naveen. 2020. *Histograms based on varying data distribution*. Hewlett Packard Enterprise Development LP. https://patents.google.com/patent/US10628442B1/en

[27] Stratos Idreos Fabian Groffen Niels Nes and Stefan Manegold Sjoerd Mullender Martin Kersten. 2012. MonetDB: Two decades of research in column-oriented database architectures. *Data Engineering* 40 (2012).

[28] Hung Q. Ngo, Ely Porat, Christopher Ré, and Atri Rudra. 2018. Worst-case Optimal Join Algorithms. *J. ACM* 65, 3, Article 16 (mar 2018), 40 pages. https://doi.org/10.1145/3180143

[29] The University of Tennessee. 2023. *PAPI: Performance Application Programming Interface*. Innovative Computing Laboratory. http://icl.utk.edu/papi/

[30] Orestis Polychroniou, Arun Raghavan, and Kenneth A. Ross. 2015. Rethinking SIMD Vectorization for In-Memory Databases. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data* (Melbourne, Victoria, Australia) *(SIGMOD '15)*. Association for Computing Machinery, New York, NY, USA, 1493–1508. https://doi.org/10.1145/2723372.2747645

[31] Mark Raasveldt and Hannes Muehleisen. [n.d.]. *DuckDB*. https://github.com/duckdb/duckdb

[32] Kenneth A. Ross. 2004. Selection Conditions in Main Memory. *ACM Trans. Database Syst.* 29, 1 (mar 2004), 132–161. https://doi.org/10.1145/974750.974755

[33] Bogdan Răducanu, Peter Boncz, and Marcin Zukowski. 2013. Micro Adaptivity in Vectorwise. In *Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data* (New York, New York, USA) *(SIGMOD '13)*. Association for Computing Machinery, New York, NY, USA, 1231–1242. https://doi.org/10.1145/2463676.2465292

[34] Tessil. 2023. *Tessil/robin-map*. https://github.com/Tessil/robin-map MIT License.

[35] Transaction Processing Performance Council. 2024. TPC-H Benchmark. https://www.tpc.org/tpch/. Accessed: 2024-05-22.

[36] Gary Scott Tyson. 1994. The Effects of Predicated Execution on Branch Prediction. In *Proceedings of the 27th Annual International Symposium on Microarchitecture* (San Jose, California, USA) *(MICRO 27)*. Association for Computing Machinery, New York, NY, USA, 196–206. https://doi.org/10.1145/192724.192753

[37] Leif Uhsadel, Andy Georges, and Ingrid Verbauwhede. 2008. Exploiting Hardware Performance Counters. In *2008 5th Workshop on Fault Diagnosis and Tolerance in Cryptography*. 59–67. https://doi.org/10.1109/FDTC.2008.19

[38] Vincent M. Weaver and Sally A. McKee. 2008. Can hardware performance counters be trusted?. In *2008 IEEE International Symposium on Workload Characterization*. 141–150. https://doi.org/10.1109/IISWC.2008.4636099

[39] Steffen Zeuch, Holger Pirk, and Johann-Christoph Freytag. 2016. Non-Invasive Progressive Optimization for in-Memory Databases. *Proc. VLDB Endow.* 9, 14 (oct 2016), 1659–1670. https://doi.org/10.14778/3007328.3007332

[40] Zuyu Zhang, Harshad Deshmukh, and Jignesh M Patel. 2019. Data Partitioning for In-Memory Systems: Myths, Challenges, and Opportunities.. In *CIDR*.

[41] Marcin Zukowski, Peter A. Boncz, Niels Nes, and Sándor Héman. 2005. MonetDB/X100 - A DBMS In The CPU Cache. *IEEE Data Eng. Bull.* 28 (2005), 17–22. https://api.semanticscholar.org/CorpusID:18052553